

Rapport TER

Approche moderne du rendu différentiel

2026

Corentin VAILLANT, Julien LEFEBVRE

Université de Toulouse

2026

L3 MIDL

Encadrant : Mathias PAULIN

Table des matières

1	Introduction	3
2	Le Rendu	3
2.1	L'équation du rendu	3
2.2	Les méthodes de Monte Carlo	4
2.3	Par quadrature : le Ray Tracing	5
2.4	Par Méthode de Monte Carlo : le Path Tracing	5
2.5	Autres Méthodes	8
3	La Différentiation	8
3.1	Differentiation par différence fini	8
3.2	Différentiation sur l'espace des chemins	9
3.3	Différentiation des estimateurs	15
4	Implantation des algorithmes	16
4.1	Premier ray tracer	16
4.2	Autres implantation CPU	16
4.3	Passage sur GPU	18
5	Conclusion	19
6	Annexe	20
	Bibliographie	21

1 | Introduction

Le rendu différentiel est une branche de l'informatique graphique où l'on cherche à trouver la différentielle de l'image d'un rendu d'une scène, ce qui est très utile dans de nombreux domaines. Il possède de nombreuses applications, notamment dans le rendu plus classique, car il permet de faire varier des paramètres de la scène sans obligation de rendre la scène une nouvelle fois de zéro, ce qui peut être très coûteux. Il est aussi utile dans le rendu physiquement réaliste parce qu'il a permis de résoudre des problèmes d'analyse-par-synthèse dans des domaines comme le rendu de vêtements ou encore la création de matériaux translucides. Son utilité ne se limite pas qu'au rendu, il possède aussi des applications dans le domaine du machine learning car il permet d'entraîner des réseaux de neurones de manière plus efficace. Le rendu différentiel a une communauté de recherche très active. En effet ce domaine est très challengeant parce qu'à ce jour aucun algorithme efficace n'introduisant pas de biais n'a été découvert. Cela est notamment dû au fait de l'absence d'estimateurs de Monte Carlo efficaces.

Dans ce TER, nous nous sommes intéressés aux travaux de *Cheng Zhang, Bailey Miller, Kai Yan, Ioannis Gkioulekas et Shuang Zhao* (**Path-Space Differentiable Rendering** [1]) qui proposent une solution sans biais à ce problème basée sur la séparation en deux sous-problèmes.

Pour effectuer ce TER, nous nous sommes aussi intéressés au ray tracing ainsi qu'au path tracing, notamment grâce à la série de livres **Ray Tracing in One Weekend — The Book Series** [2], [3], [4], dans le but d'acquérir les bases nécessaires à la compréhension du papier.

Dans le cadre de ce travail, nous avons aussi tenté d'implanter un Path Tracer sur GPU avec Vulkan (**Github vers les projet**).

Nous allons introduire dans un premier temps le ray tracing ainsi que ses limites. Puis nous regarderons ce qu'est le Path Tracing et comment il corrige les limites du Ray Tracing. Dans un troisième temps, nous parlerons du rendu différentiel et de la méthode proposée par *Shuang Zhao* et son équipe. Enfin nous finirons par regarder une autre approche à ce problème avec les travaux de *Tizian Zeltner, Sébastien Speierer, Iliyan Georgiev et Wenzel Jakob* (**Monte Carlo Estimators for Differential Light Transport** [5]).

2 | Le Rendu

2.1 | L'équation du rendu

Pour effectuer un rendu 3D, nous allons essayer de calculer la luminance des différentes surfaces de notre scène. La luminance représente la luminosité que l'œil humain perçoit, provenant d'une surface qui émet de la lumière soit en tant que source de lumière, soit par réflexion ou transmission. L'équation du rendu (**The rendering equation** [6]) nous permet de calculer la luminance sur les différentes surfaces de notre scène 3D. On peut la définir comme ci-dessous :

Définition 2.1. (Equation du Rendu)

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} L_i(x, \omega_i) f_i(x) \overline{\cos(\theta_i)} d\omega_i \quad (1)$$

- $L_o(x, \omega_o)$ représente la luminance sortante au point x dans la direction ω_o .
- $L_e(x, \omega_o)$ représente la luminance émise par le point x dans la direction ω_o .
- Ω représente la sphère de rayon 1 autour du point x .
- $L_i(x, \omega_i)$ représente la luminance arrivant en x depuis la direction ω_i et elle est définie de la manière suivante : $L_i(x, \omega_i) = L_o(vis(x, \omega_i), -\omega_i)$ où $vis(x, \omega_i)$ donne le premier point intersecté en partant dans la direction ω_i à partir du point x .
- $f_i(x)$ représente la BSDF (Bidirectional Scattering Distribution Function) qui nous donne la distribution de la luminance arrivant en x depuis la direction ω_i et dispersé dans la direction ω_o , il s'agit en fait d'une fonction décrivant les propriétés photométriques d'une matière.
- θ représente l'angle formé ω_i et la $\vec{n}$ normale de la surface.
- $\overline{\cos(\theta)} = \cos(\theta)$ si $\cos(\theta) > 0$ sinon 0.

L'un des problèmes de cette équation est que l'on ne peut pas résoudre l'intégrale de façon analytique, notamment en raison de sa nature récursive. C'est pour cela que des méthodes pour estimer cette intégrale ont été mises en place.

2.2 | Les méthodes de Monte Carlo

En informatique, lorsqu'il s'agit de résoudre un problème impliquant le calcul d'une valeur numérique, nous utilisons souvent des méthodes dites de « Monte Carlo », en référence au célèbre casino qui s'y trouve. L'idée est d'approcher une valeur numérique à l'aide de procédés aléatoires empiriques. Cela s'avère particulièrement utile pour calculer des intégrales sur des domaines non triviaux ($\mathbb{R}^n$, espace des chemins, ...).

Voici comment approcher la valeur d'une intégrale grâce aux méthodes de Monte Carlo :

Soit φ une fonction continue par morceaux sur Ω , on note $I := \int_{\Omega} \varphi(x) dx$, et on suppose que I converge.

On dispose d'une variable aléatoire U de densité f et de support Ω (i.e. $\int_{\Omega} f(x) dx = 1$).

On cherche à calculer $I = \mathbb{E}(\varphi(U)/f(U))$. Cette quantité peut être approchée de manière empirique grâce au théorème du transfert :

$$\overline{\varphi(U)}_N = \frac{1}{N} \sum_{i=1}^N \frac{\varphi(U(\omega_i))}{f(U(\omega_i))},$$

où $(\omega_i)_{1 \leq i \leq N} \in \text{supp}(U)^N$ et $\text{supp}(U)$ désigne le support de U .

La loi des grands nombres nous permet d'avoir que cet estimateur converge presque sûrement vers I lorsque $N \rightarrow +\infty$:

$$\overline{\varphi(U)}_N \xrightarrow[N \rightarrow +\infty]{} I$$

Le choix de f nous permet d'obtenir différentes vitesses de convergences, en effet, si f est concentré là où φ est grand, la variance de l'estimateur sera réduite, et donc la convergence plus rapide.

Il faut néanmoins noter que si $\text{supp}(U) \subsetneq \Omega$, l'estimateur sera biaisé, en effet, certaines régions de l'espace d'intégrations ne seront jamais explorées, et donc leurs valeurs jamais ajoutées au résultat de l'intégrale.

2.3 | Par quadrature : le Ray Tracing

Définition 2.2. (Rayon) On appelle rayon un couple $\vec{r} = (o, \vec{d}) \in \mathbb{R}^3 \times \mathbb{U}(\mathbb{R}^3)$ où o est un point désignant l'origine du rayon et $\vec{d}$ est un vecteur unitaire désignant la direction du rayon. À noter que la direction n'est pas nécessairement unitaire, mais nous la prendrons ainsi sans perte de généralité pour des raisons de simplicité dans le reste de ce rapport.

Le ray tracing (lancer de rayons dans la langue de Molière) est un algorithme permettant d'estimer l'équation du rendu (équ. 1). Il se base sur une technique de lancer de rayons à partir de la caméra vers la scène, c'est-à-dire dans le sens inverse de la lumière. Cela donne le même résultat que dans le sens de la lumière d'après le principe du retour inverse de la lumière de Fermat. Il se base aussi sur les lois de Snell-Descartes de réflexion et réfraction de la lumière. Il s'agit en fait d'une quadrature de la scène, à partir de rayons envoyés récursivement. Une méthode de ray tracing naïve de la marche aléatoire :

Algorithm 1 . – Marche Aléatoire

```

1: procedure  $L_i(\text{Rayon } \vec{r}, \text{depth} \in \mathbb{N})$ 
2:    $\text{intersection} \leftarrow \text{Intersection}(\vec{r})$ 
3:    $\triangleright$  Cas où le rayon n'intersecte aucune surface
4:   if  $\neg \text{intersection}$  then
5:     return  $\text{LumièresDirectionelles}.L_e(\vec{r})$ 
6:   end
7:    $\triangleright$  Recupération de la surface intersectée et de son émission
8:    $\text{surface} \leftarrow \text{intersection.surface}$ 
9:    $\omega_o \leftarrow -\vec{r}.\vec{d}$ 
10:   $L_e \leftarrow \text{surface}.Le(\omega_o)$ 
11:   $\triangleright$  Cas où on a atteint le nombre maximum d'itération
12:  if  $\text{depth} = \text{maxDepth}$  then
13:    return  $L_e$ 
14:  end
15:   $\triangleright$  Calcul de la partie non récursive de l'intégrale de l'équation du rendu
16:   $\text{bsdf} \leftarrow \text{surface.BSDF}$ 
17:   $\omega_i \leftarrow \mathcal{U}(\Omega)$ 
18:   $f_{\cos} \leftarrow \text{bsdf}(\omega_o, \omega_i) \times |\omega_i \cdot \vec{n}|$ 
19:   $\triangleright$  Si la partie non récursive est nulle il est inutile de continuer d'itérer
20:  if  $f_{\cos} = 0$  then
21:    return  $L_e$ 
22:  end
23:   $\triangleright$  Appel récursif,  $(\frac{1}{4\pi}$  : Probabilité de tiré une direction.)
24:   $\vec{r} \leftarrow (\text{intersection.point}, \omega_i)$ 
25:  return  $L_e + f_{\cos} \times L_i(\vec{r}, \text{depth} + 1) \div (\frac{1}{4\pi})$ 
26: end
```

Généralement, les implantations du lancer de rayons sont avec de meilleurs moyens pour estimer le prochain rayon que la marche aléatoire, afin d'obtenir une meilleure convergence.

2.4 | Par Méthode de Monte Carlo : le Path Tracing

L'équation du rendu (équ. 1) peut être réécrite de façon à enlever son aspect récursif. Pour cela, au lieu d'intégrer sur la sphère unité autour de chacun des points, nous allons effectuer un

changement de variable et intégrer sur des chemins. Cette réécriture a été proposée par *Veach* en 1997, notamment dans sa thèse **Robust monte carlo methods for light transport simulation** [7].

Définition 2.3. (Chemin de lumière et espace des chemins) Soit $\mathcal{M}$ l'ensemble des surfaces des objets de notre scène. On appelle chemin de lumière (ou light path) un vecteur $\bar{x} = (x_0, x_1, \dots, x_N)$ de $\mathcal{M}^{N+1}$. Les chemins commencent sur une lumière en x_0 pour aller jusqu'à la caméra en x_N . On appelle espace des chemins (ou path space) l'ensemble $\Omega := \cup_{N=1}^{\infty} \mathcal{M}^{N+1}$.

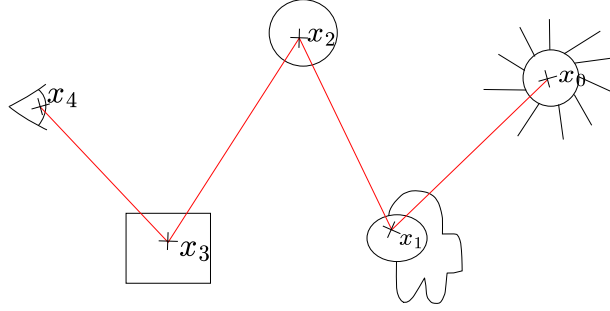


Fig. 2 . – Exemple de chemin de $\mathcal{M}^{N+1}$

En réécrivant l'équation du rendu (éq. 1) sur l'espace des chemins, on obtient:

Définition 2.4. (Équation du rendu sur l'espace des chemins)

$$I = \int_{\Omega} f(\bar{x}) d\mu(\bar{x}) \quad (2)$$

Où μ est la mesure du produit des aires défini par $d\mu(\bar{x}) := \prod_{n=0}^N dA(x_n)$ avec A la mesure de l'aire d'une surface et $f(\bar{x})$ est défini de la façon suivante :

$$f(\bar{x}) = \left(\prod_{n=0}^{N-1} g(x_{n+1} : x_n, w_n) \right) \cdot W_e(x_N \rightarrow x_{N-1}) \quad (3)$$

où W_e est la sensibilité du capteur, dans notre cas, W_e sera une constante égale à 1 car on utilise une caméra trou d'épingle et

$$g(x_{n+1} : x_n, w_n) := f_s(x_n \rightarrow x_{n+1}) \cdot G(x_n \leftrightarrow x_{n+1}) \quad (4)$$

où f_s est la BSDF au point x_n dans la direction arrivant de x_{n-1} et allant vers x_{n+1} si $n > 0$ sinon $f_s := L_e(x_0 \rightarrow x_1)$ où L_e est l'émission de la surface x_0 dans la direction de x_1 et

$$G(x_n \leftrightarrow x_{n+1}) := \mathbb{V}(x_n \leftrightarrow x_{n+1}) G_0(x_n \leftrightarrow x_{n+1}) \quad (5)$$

où $\mathbb{V}(x_n \leftrightarrow x_{n+1})$ vaut 1 si x_n et x_{n+1} sont visibles, c'est-à-dire s'il n'y a pas de surfaces opaques entre eux, et 0 sinon et

$$G_0(x_n \leftrightarrow x_{n+1}) := \frac{|\vec{n}_{x_n} \cdot \omega_n| |\vec{n}_{x_{n+1}} \cdot (-\omega_n)|}{\|x_{n+1} - x_n\|^2} \quad (6)$$

où ω_n représente le vecteur unitaire partant de x_n et pointant vers x_{n+1} .

Cette réécriture de l'équation du rendu sur l'espace des chemins est ce qui nous permettra dans la suite, lorsque nous aborderons les travaux de *Shuang Zhao* et son équipe (**Path-Space**

Differentiable Rendering [1]) de calculer la différentielle de notre scène. En effet, on verra qu'il est possible de « rentrer » la différentielle à l'intérieur de l'intégrale et que cela nous créera deux intégrales, une intérieure (ou interior) et une de bordure (ou boundary).

Cette réécriture est aussi à la base de l'algorithme du path tracing, qui est fondamental car nous verrons qu'il est possible de le différencier. Il se base sur une méthode de Monte Carlo pour estimer l'intégrale sur les chemins (équ. 2).

Algorithm 2 . – Path Tracer

```

1: procedure  $L_i(\text{Rayon } \vec{r})$ 
2:    $\omega_o \leftarrow \vec{r} \cdot \vec{d}$ 
3:    $x_1 \leftarrow \vec{r} \cdot o$ 
4:   intersection  $\leftarrow \text{Intersection}(\vec{r})$ 
5:   if  $\neg \text{intersection}$  then
6:     return LumièresDirectionelles. $L_e(\vec{r})$ 
7:   end
8:    $x_2 \leftarrow \text{intersection.point}$ 
9:    $L \leftarrow \text{intersection}.L_e(\omega_o)$ 
10:   $\beta \leftarrow 1$ 
11:   $\triangleright \varepsilon > 0$  fixé.
12:  while  $\beta > \varepsilon$  do
13:     $\triangleright$  Éclairage direct (échantillonnage de la lumière)
14:     $p_0 \leftarrow p \sim \mathbb{P}_{\text{lumières}}$ 
15:     $\omega_i \leftarrow \frac{p_0 - x_2}{\|p_0 - x_2\|}$   $\triangleright$  direction de  $x_2$  vers  $p_0$ 
16:     $\alpha_{\text{directe}} \leftarrow p_0 \cdot L_e(p_0 \rightarrow x_1) f_s(p_0 \rightarrow x_1 \rightarrow x_2) G(p_0, x_1)$ 
17:     $L \leftarrow L + \beta \alpha_{\text{directe}} / \mathbb{P}_{\text{lumières}}(p_0)$ 
18:
19:     $\omega_i \leftarrow \omega \sim \mathbb{P}_{\text{bsdf}}$ 
20:     $\triangleright (x_1, \omega_i)$  : le rayon partant de  $x_1$  avec la direction  $\omega_i$ 
21:    intersection  $\leftarrow \text{Intersection}((x_1, \omega_i))$ 
22:    if  $\neg \text{intersection}$  then
23:      BREAK
24:    end
25:     $\alpha_{\text{indirecte}} \leftarrow f_s(x_0 \rightarrow x_1 \rightarrow x_2) G(x_0, x_1)$ 
26:     $\triangleright$  On convertie la probabilité en mesure d'aire :
27:     $q \leftarrow \mathbb{P}_{\text{bsdf}}(\omega_i) \frac{|\mathbf{n}(x_0) \cdot -\omega_i|}{\|x_0 - x_1\|^2}$ 
28:     $\beta \leftarrow \beta \alpha_{\text{indirecte}} / q$ 
29:
30:     $\triangleright$  On continue le chemin.
31:     $(x_2, x_1) \leftarrow (x_1, x_0)$ 
32:     $\vec{r} \leftarrow (x_1 \rightarrow x_2)$ 
33:  end
34:  return  $L$ 
35: end

```

Ici, le ε utilisé dans la boucle introduit un biais. L'idée derrière celui-ci est que lorsque le poids d'un chemin devient trop faible (β), l'ajout d'illuminations à l'accumulateur L devient négligeable, le biais est introduit par le fait que tous les chemins de poids strictement supérieur à 0 ne peuvent être explorés, nous prendrons donc un sous-ensemble de notre espace d'intégrations. Le contrôle du ε nous permet d'ajuster le biais en fonction du résultat souhaité : plus ε sera

proche de 0, moins le biais sera présent.

Afin de retirer le biais, nous pouvons mettre en place une méthode dite de « roulette russe », qui consiste à fixer une probabilité de terminer le chemin en fonction de β . Plus β est proche de 0, plus nous avons de chances de terminer le chemin. Cela nous permet de toujours avoir la possibilité d'explorer tous les chemins possibles.

2.5 | Autres Méthodes

De nombreuses autres méthodes existent pour résoudre l'équation du rendu et ont chacune leurs avantages mais aussi leurs inconvénients. Parmi celles-ci, on retrouve :

1. La rasterisation qui, comme le ray tracing, est une quadrature de l'équation du rendu. Pour le rendu en temps réel, cette méthode est souvent utilisée, bien que le ray tracing soit de plus en plus mis en avant. Elle est non biaisée et différentiable. Mais elle reste très limitée, notamment au niveau de la différenciation et a une convergence assez lente.
2. Le photon mapping (cartographie des photons) qui se base aussi sur une technique de lancer de rayon. Contrairement au ray tracing et au path tracing, les rayons sont envoyés depuis les lumières et on enregistre leurs chemins dans une carte de photons. Puis pour effectuer le rendu, on va envoyer un rayon depuis la caméra et à son point d'intersection avec une surface, regarder la concentration de points à proximité sur la carte de photons avec un algorithme des plus proches voisins. Cet algorithme est biaisé, donc il n'a pas vraiment d'intérêt à être différencié, mais il joue un rôle dans la méthode proposée par *Shuang Zhao* et son équipe (**Path-Space Differentiable Rendering** [1]), nous en reparlerons dans la section dédiée.
3. Le bidirectional path tracing est une version améliorée du path tracing. Au lieu de créer des chemins uniquement depuis la caméra, il va aussi créer des chemins partant des lumières et les connecter entre eux avec un rayon de visibilité. De plus cet algorithme est non biaisé mais, malheureusement, à ce jour aucune technique de différenciation n'a été découverte.
4. Bien d'autres méthodes existent, comme le **Metropolis Ligth Transport** [8] qui se base sur l'algorithme de Metropolis-Hastings, le **Vertex Connection and Merging** [9] qui fusionne le bidirectional path tracing et le photon mapping, le **Cone Tracing** [10] qui donne une épaisseur aux rayons, ce que ne fait pas le raytracing ou encore le **Gaussian Splating** [11] qui effectue le rendu à l'aide de données extraites d'image et beaucoup d'autres...

3 | La Différentiation

3.1 | Differentiation par différence fini

Pour obtenir une approximation, nous pourrions utiliser la méthode des différences finies.

Pour toute fonction $\mathcal{C}^1 f$, le théorème de Taylor nous donne pour V un voisinage de x_0 :

$$\begin{aligned}
 h &: x_0 + h \in V \\
 f(x_0 + h) &= f(x_0) + f'(x_0)h + R(x_0 + h) : \text{Avec } R(x_0 + h) \text{ le reste.} \\
 \Leftrightarrow \frac{f(x_0+h) - f(x_0)}{h} &= f'(x_0) + \frac{R(x_0+h)}{h} \\
 \Leftrightarrow f'(x_0) &= \lim_{h \rightarrow 0} \frac{f(x_0+h) - f(x_0)}{h}
 \end{aligned}$$

En assumant $R(x_0 + h)$ suffisamment petit, nous avons donc :
 $f'(x_0) \simeq \frac{f(x_0+h)-f(x_0)}{h}$

Nous pouvons donc appliquer cette méthode, en utilisant h assez petit.

Ici $\pi \in \mathbb{R}$ représente un paramètre de la scène. On notera toutes les fonctions dépendant de la scène à l'aide de la syntaxe suivante $\varphi(\pi : x_1, \dots, x_k)$, où φ est une fonction quelconque dépendant des paramètres de la scène.

Nous avons donc :

$$\frac{\partial}{\partial \pi} \int_{\Omega} f(\pi : \bar{x}) d\mu(\bar{x}) \simeq \frac{\int_{\Omega} f(\pi : \bar{x}) d\mu(\bar{x}) - \int_{\Omega} f(\pi-h : \bar{x}) d\mu(\bar{x})}{h}$$

La méthode est donc facile à comprendre et à implanter, mais elle souffre de deux gros inconvénients. Premièrement, la stabilité numérique des nombres à virgule flottante. En effet, si l'on prend h trop petit, les imprécisions dans le calcul deviendront de plus en plus importantes.

Et de plus, cette méthode nécessite de faire deux rendus, ce qui provoque donc le doublage du temps de calcul.

3.2 | Différentiation sur l'espace des chemins

Avant de commencer à aborder la méthode de *Shuang Zhao* et son équipe (**Path-Space Differentiable Rendering** [1]) nous allons d'abord introduire quelques définitions importantes.

Définition 3.1. (Configuration de référence, mouvement et déformation) Soit $\mathcal{B}$ un manifold 2D abstrait que l'on nommera configuration de référence. On appelle une déformation de $\mathcal{B}$ une fonction injective différentiable de $\mathcal{B}$ sur une surface $\mathcal{M}$. On appelle X un mouvement de $\mathcal{B}$ une fonction $\mathcal{C}^3$ de $\mathcal{B} \times \mathbb{R}$ dans $\mathcal{M}$. De plus nous pouvons remarquer que si on fixe un paramètre $\pi \in \mathbb{R}$, alors $X(\cdot, \pi)$ est une déformation de $\mathcal{B}$.

Définition 3.2. (Surface évoluant, carte de référence et trajectoire) Pour un mouvement X et un paramètre π , on appelle surface évoluant l'image $\mathcal{M}(\pi) := \{X(p, \pi) : p \in \mathcal{B}\} \in \mathbb{R}^3$ de la fonction $X(p, \pi)$. De plus, comme cette fonction est injective, on peut définir une fonction inverse sur son image $P(\cdot, \pi) : \mathcal{M}(\pi) \rightarrow \mathcal{B}$ que l'on appellera carte de référence. Enfin on appellera trajectoire l'ensemble $\mathcal{T}$ des couples $(x, \pi) \in \mathcal{M}(\pi) \times \mathbb{R}$ des surfaces évoluant et de leur paramètre π associé.

Définition 3.3. (Espace des matériaux et espace des position) On appellera $p \in \mathcal{B}$ un point de matériau et $x \in \mathcal{M}(\pi)$ un point de position. Nous pouvons aussi remarquer que la déformation $X(\cdot, \pi)$ établit une bijection entre les points de matériau et les points de position. On appellera champ de matériau une fonction de $\mathcal{B} \times \mathbb{R}$ et champ de position une fonction sur la trajectoire $\mathcal{T}$.

Définition 3.4. (Paramétrisation local) Pour une trajectoire $\mathcal{T}$ nous pouvons définir une paramétrisation locale, proche de $(x, \pi) \in \mathcal{T}$. On fixe $\hat{x}(\xi, \pi')$, qui pour un ouvert $\mathcal{O} \in \mathbb{R}^2$:

- $\hat{x}(\mathcal{O}, \pi) \subset \mathcal{M}(\pi)$
- $\forall \pi'$ proche de π : $\hat{x}(\cdot, \pi')$ est $\mathcal{C}^1$ et injectif.
- $\forall \xi \in \mathcal{O}$: $\hat{x}(\xi, \cdot)$ est $\mathcal{C}^1$ sur un voisinage de π .

Définition 3.5. (Vitesse) Avec $x \in \mathcal{M}(\pi)$ et la paramétrisation $\hat{x}$, il existe un unique $\xi \in \mathcal{O}$ qui satisfait $\hat{x}(\xi, \pi) = x$. On note ξ comme étant les coordonnées locales de x . On peut donc définir la vitesse local de x comme :

$$v(x, \pi) = \frac{\partial \hat{x}(\xi, \pi')}{\partial \pi'} \Big|_{\pi'=\pi} \quad (7)$$

Et donc en utilisant $\mathcal{M}(\pi)$ orienté par un champ de vecteurs normaux $\boldsymbol{n}(x, \pi)$, on définit $V = v \cdot \boldsymbol{n}$ et $v_{\text{tan}} = v - V\boldsymbol{n}$, respectivement, la vitesse scalaire locale et la vitesse tangentielle locale.

Pour les définitions, nous allons poser $\varphi(x, \pi)$ un champ de position scalaire sur $\mathcal{M}(\pi)$.

Définition 3.6. (Dérivée de scène) φ admet une dérivée de scène de la forme :

$$\dot{\varphi}(x, \pi) = \frac{\partial}{\partial \pi'} \varphi(\hat{x}(\xi, \pi'), \pi') \Big|_{\pi'=\pi} \quad (8)$$

Elle admet aussi une forme normalisée de cette dérivée :

$$\varphi^\square = \dot{\varphi} - v_{\text{tan}} \cdot \text{grad}_{\mathcal{M}}(\varphi) \quad (9)$$

Définition 3.7. (Courbes de discontinuité et bordure étendue) La fonction φ est $\mathcal{C}^0$ en fonction de x sauf sur un ensemble de courbes qui évolue de façon continue en fonction de π . On pose $\Delta\mathcal{M}[\varphi](\pi) \subset \mathcal{M}(\pi)$ l'ensemble de ces courbes de discontinuité. On appellera $\overline{\partial\mathcal{M}}[\varphi](\pi)$ la bordure étendue l'ensemble des bordures de $\mathcal{M}(\pi)$ et des courbes de discontinuité.

En utilisant la relation de transport venant de la mécanique des fluides par *Cermelli* (**Transport relations for surface integrals arising in the formulation of balance laws for evolving fluid interfaces** [12]) on peut obtenir :

$$\frac{d}{d\pi} \int_{\mathcal{M}} \varphi dA = I_{\text{intérieur}} + I_{\text{bordure}} \quad (10)$$

Avec

$$I_{\text{intérieur}} = \int_{\mathcal{M}} (\varphi^\square - \varphi \kappa V) dA$$

$$I_{\text{bordure}} = \int_{\overline{\partial\mathcal{M}}} \Delta\varphi V_{\overline{\partial\mathcal{M}}} d\ell$$

Avec ℓ la mesure de la longueur des courbes, κ la courbure totale, V et $V_{\overline{\partial\mathcal{M}}}$ les vitesses normales scalaires de $\mathcal{M}(\pi)$ et de la bordure étendue et $\Delta\varphi(x, \pi)$ le prolongement défini comme ci-dessous :

$$\Delta\varphi(x, \pi) := \begin{cases} \varphi(x, \pi) & \text{si } x \in \partial\mathcal{M}(\pi) \\ \varphi^-(x, \pi) - \varphi^+(x, \pi) & \text{si } x \in \Delta\mathcal{M}(\pi) \end{cases}$$

Nous pouvons, dans le cas particulier où $\mathcal{M}$ est indépendant des paramètres de la scène, simplifier cette relation (éq. 10) à la relation de transport standard de *Reynolds* (**The Sub-Mechanics of the Universe** [13]) :

$$\frac{d}{d\pi} \int_{\mathcal{M}} \varphi dA = \int_{\mathcal{M}} \dot{\varphi} dA + \int_{\Delta\mathcal{M}} \Delta\varphi V_{\Delta\mathcal{M}} d\ell \quad (11)$$

Nous allons par la suite essayer d'appliquer cette relation (éq. 11) à l'équation du rendu (éq. 2). Nous allons d'abord nous concentrer sur le cas de l'éclairage direct. Pour ce faire,

nous allons nous placer dans une configuration où une surface $\mathcal{M}_{\text{obj}}$ est éclairée par une surface évolutive $\mathcal{L}(\pi)$. Dans cette configuration pour $y, y' \in \mathcal{M}_{\text{obj}}$ on a :

$$I_{\text{direct}} = \int_{\mathcal{L}} f_{\text{direct}}(x) dA(x) \quad (12)$$

avec $f_{\text{direct}}(x) := L_e(x \rightarrow y) f_s(x \rightarrow y \rightarrow y') G(x \leftrightarrow y)$

Pour simplifier la dérivation, nous allons poser les axiomes suivants :

A.1 Pour tout $x \in \mathcal{L}(\pi)$ il existe une paramétrisation tel que x possède une vitesse tangentielle nulle.

A.2 $L_e(x \rightarrow y) f_s(x \rightarrow y \rightarrow y')$ est continue par rapport à x lorsque $y, y' \in \mathcal{M}_{\text{obj}}$ sont fixées.

Sous ces deux axiomes en appliquant éq. 10 à f_{direct} on obtient :

$$\frac{\partial I_{\text{direct}}}{\partial \pi} = \int_{\mathcal{L}} |\dot{f}_{\text{direct}} - f_{\text{direct}} \kappa V| dA + \int_{\partial \mathcal{L}} \Delta f_{\text{direct}} V_{\partial \mathcal{L}} d\ell \quad (13)$$

En effet, grâce à l'axiome **A.1** nous avons que $(f_{\text{direct}})^\square = \dot{f}_{\text{direct}}$. De plus, sous l'effet de l'axiome **A.2** nous avons que :

$$\Delta f_{\text{direct}} = L_e(x \rightarrow y) f_s(x \rightarrow y \rightarrow y') \Delta G(x \leftrightarrow y) \quad (14)$$

Un des problèmes de cette relation (éq. 13) est qu'elle est très coûteuse à estimer dans des scènes complexes. C'est pour cela que nous allons nous placer dans l'espace des matériaux à l'aide d'un changement de variable. Nous obtenons ainsi :

$$I_{\text{direct}} = \int_{\mathcal{B}} \hat{f}_{\text{direct}}(p) dA(p) \quad (15)$$

avec $\hat{f}_{\text{direct}} := f_{\text{direct}}(x) J(p)$ avec $x = X(p, \pi)$ et J le déterminant de la Jacobienne du changement de variable défini comme $J(p) = |dA(x)/dA(p)|$.

Comme nous intégrons sur la configuration de référence $\mathcal{B}$ qui ne dépend pas du paramètre π nous pouvons appliquer la relation de *Reynolds* (éq. 11) ce qui nous donne :

$$\frac{\partial I_{\text{direct}}}{\partial \pi} = \int_{\mathcal{B}} (\hat{f}_{\text{direct}})^\cdot dA + \int_{\Delta \mathcal{B}} \Delta \hat{f}_{\text{direct}} V_{\Delta \mathcal{B}} d\ell \quad (16)$$

Nous allons maintenant essayer de généraliser cette relation. Pour ce faire, nous allons utiliser le fait que $I = \sum_{N=1}^{\infty} I_N$ où I_N est l'intégrale du rendu (éq. 2) restreinte aux chemins de taille N . Nous allons commencer par réécrire I_N de façon récursive. Pour ce faire, nous allons poser les fonctions h_n telles que :

$$h_N(x_N; x_{N-1}) := W_e(x_N \rightarrow x_{N-1}) \quad (17)$$

et pour tout $0 \leq n < N$:

$$h_n := \int_{\mathcal{M}} g(x_{n+1}; x_{n-1}, x_n) h_{n+1}(x_{n+1}; x_n) dA(x_{n+1}) \quad (18)$$

Ce qui permet d'obtenir :

$$h_0(x_0) = \int_{\mathcal{M}^N} f(\bar{x}) \prod_{n=1}^N dA(x_n) \quad (19)$$

$$I_N = \int_{\mathcal{M}} h_0(x_0) dA(x_0) \quad (20)$$

Nous pouvons appliquer à h_n la relation de transport (éq. 10) ce qui nous donne :

$$\dot{h}_n = \int_{\mathcal{M}} [(h_{n+1}g)^\cdot - h_{n+1}g\kappa V] dA + \int_{\overline{\partial\mathcal{M}}_{n+1}} h_{n+1} \Delta g V_{\overline{\partial\mathcal{M}}_{n+1}} d\ell \quad (21)$$

Comme pour le cas direct dans l'espace des positions, grâce à l'axiome **A.1** nous avons que $(h_{n+1}g)^\square = (h_{n+1}g)^\cdot$.

Nous pouvons ainsi différencier I_N en développant successivement les h_n et $\dot{h}_n$ et nous obtenons ainsi :

$$\frac{\partial I_N}{\partial \pi} = \int_{\Omega_N} \left[\dot{f}(\bar{x}) - f(\bar{x}) \sum_{K=0}^N \kappa(x_K) V(x_K) \right] d\mu(\bar{x}) \quad (22(1))$$

$$+ \sum_{K=0}^N \left[\int_{\partial\Omega_{N,K}} \Delta f_K(\bar{x}) V_{\overline{\partial\mathcal{M}}_K}(x_K) d\mu'_{N,K}(\bar{x}) \right] \quad (22(2))$$

où :

$$\partial\Omega_{N,K} := \underbrace{M(\pi)^K}_{\text{chemin vers la lumière}} \times \underbrace{\overline{\partial\mathcal{M}}_K(\pi)}_{\text{point de discontinuité}} \times \underbrace{\mathcal{M}(\pi)^{N-K}}_{\text{chemin vers la caméra}} \quad (23)$$

$$d\mu'_{N,K} := d\ell(x_K) \prod_{\substack{0 \leq n \leq N \\ n \neq K}} \quad (24)$$

$$\Delta f_K(\bar{x}) = \frac{f(\bar{x}) \Delta g(x_K; x_{K-2}, x_{K-1})}{g(x_K; x_{K-2}, x_{K-1})} \quad (25)$$

Ainsi nous pouvons sommer la différentielle des I_N (éq. 22) pour obtenir la différentielle de I on a ainsi :

$$\frac{\partial I}{\partial \pi} = \int_{\Omega} \left[\dot{f}(\bar{x}) - f(\bar{x}) \sum_{K=0}^N \kappa(x_K) V(x_K) \right] d\mu(\bar{x}) \quad (26(1))$$

$$+ \int_{\partial\Omega} \Delta f_K(\bar{x}) V_{\overline{\partial\mathcal{M}}_K}(x_K) d\mu'(\bar{x}) \quad (26(2))$$

où $\partial\Omega = \cup_{N=1}^{\infty} \cup_{K=0}^N \partial\Omega_{N,K}$ et $\mu'(D) := \sum_{N=1}^{\infty} \sum_{K=0}^N \mu'_{N,K}(D \cap \partial\Omega_{N,K})$.

Comme pour le cas direct, nous pouvons effectuer un changement de variable pour nous placer dans l'espace des matériaux. Nous allons d'abord passer l'intégrale du rendu (éq. 2) dans l'espace des matériaux en effectuant un changement de variable :

$$I = \int_{\hat{\Omega}} \hat{f}(\bar{p}) d\mu(\bar{p}) \quad (27)$$

avec $\hat{\Omega} := \cup_{N=1}^{\infty} \mathcal{B}^{N+1}$ et :

$$\hat{f}(\bar{p}) = \left(\prod_{n=0}^{N-1} \hat{g}(p_{n+1}; p_{n-1}, p_n) \right) \widehat{W}_e(p_N \rightarrow p_{N-1}) \quad (28)$$

où :

$$\widehat{W}_e(p_N \rightarrow p_{N-1}) := J(p_N) W_e(p_N \rightarrow p_{N-1}) \quad (29)$$

et

$$\hat{g}(p_{n+1}; p_{n-1}, p_n) = \underbrace{\hat{f}_s(p_{n-1} \rightarrow p_n \rightarrow p_{n+1})}_{J(p_n) f_s(x_{n-1} \rightarrow x_n \rightarrow x_{n+1})} G(x_n \leftrightarrow x_{n+1}) \quad (30)$$

où J est le déterminant de la Jacobienne $J(p_n) = |dA(x_n)/dA(p_n)|$

On obtient ainsi :

$$\frac{\partial I}{\partial \pi} = \int_{\hat{\Omega}} (\hat{f})'(\bar{p}) d\mu(\bar{p}) + \int_{\partial \hat{\Omega}} \Delta \hat{f}_K(\bar{p}) V_{\Delta \mathcal{B}_K}(p_K) d\mu'(\bar{p}) \quad (31)$$

Nous allons maintenant voir les estimateurs de Monte Carlo permettant d'estimer l'intégrale intérieure et l'intégrale de bordure. Pour l'intégrale intérieure, du fait de sa ressemblance avec l'intégrale du rendu, nous pouvons construire notre estimateur sur la base de l'algorithme du path tracer :

Algorithm 3 . – Differentiation du Path Tracer : intérieur

```

1: procedure PT_diff_intérieur(Rayon  $\vec{r}$ )
2:    $\omega_o \leftarrow \vec{r} \cdot \vec{d}$ 
3:    $x_1 \leftarrow \vec{r} \cdot o$ 
4:   intersection  $\leftarrow$  Intersection( $\vec{r}$ )
5:   if  $\neg$ intersection then
6:     return LumièresDirectionelles. $L_e(\vec{r})$ 
7:   end
8:    $x_2 \leftarrow$  intersection.point
9:    $(\vec{L}, \dot{\vec{L}}) \leftarrow (\text{intersection}.L_e(\omega_o), [\text{intersection}.L_e(\omega_o)]')$ 
10:   $\beta, \dot{\beta} \leftarrow (1, 0)$ 
11:   $\triangleright \varepsilon > 0$  fixé.
12:  while  $\beta > \varepsilon$  do
13:     $\triangleright$  Éclairage direct (échantillonnage de la lumière)
14:     $p_0 \leftarrow p \sim \mathbb{P}_{\text{lumières}}$ 
15:     $x_0 \leftarrow X(p_0, \pi)$ 
16:     $\alpha_{\text{directe}} \leftarrow p_0 \cdot L_e(p_0 \rightarrow x_1) f_s(p_0 \rightarrow x_1 \rightarrow x_2) G(p_0, x_1) J(p_0)$ 
17:     $\dot{\alpha}_{\text{directe}} \leftarrow [p_0 \cdot L_e(p_0 \rightarrow x_1) f_s(p_0 \rightarrow x_1 \rightarrow x_2) G(p_0, x_1) J(p_0)]'$ 
18:     $\vec{L} \leftarrow \vec{L} + \beta \alpha_{\text{directe}} / \mathbb{P}_{\text{lumières}}(p_0)$ 
19:     $\dot{\vec{L}} \leftarrow \dot{\vec{L}} + (\beta \dot{\alpha}_{\text{directe}} + \beta \alpha_{\text{directe}}) / \mathbb{P}_{\text{lumières}}(p_0)$ 
20:
21:     $\omega_i \leftarrow \omega \sim \mathbb{P}_{\text{bsdf}}$ 
22:     $\triangleright (x_1, \omega_i)$  : le rayon partant de  $x_1$  avec la direction  $\omega_i$ 
23:    intersection  $\leftarrow$  Intersection( $(x_1, \omega_i)$ )
24:    if  $\neg$ intersection then

```

```

25:     BREAK
26:   end
27:    $\alpha_{\text{indirecte}} \leftarrow f_s(x_0 \rightarrow x_1 \rightarrow x_2)G(x_0, x_1)$ 
28:    $\dot{\alpha}_{\text{indirecte}} \leftarrow [f_s(x_0 \rightarrow x_1 \rightarrow x_2)G(x_0, x_1)]$ 
29:   ▷ On convertie la probabilité en mesure d'aire :
30:    $q \leftarrow \mathbb{P}_{\text{bsdf}}(\omega_i) \frac{|\mathbf{z}(x_0) \cdot \omega_i|}{\|x_0 - x_1\|^2}$ 
31:    $\beta \leftarrow \beta \alpha_{\text{indirecte}} / q$ 
32:    $\dot{\beta} \leftarrow (\beta \dot{\alpha}_{\text{indirecte}} + \dot{\beta} \alpha_{\text{indirecte}}) / q$ 
33:
34:   ▷ On continue le chemin.
35:    $(x_2, x_1) \leftarrow (x_1, x_0)$ 
36:    $\vec{\mathbf{r}} \leftarrow (x_1 \rightarrow x_2)$ 
37:   end
38:   return  $(L, \dot{L})$ 
39: end

```

Définition 3.8. (Sous chemin de source et sous chemin de caméra) Soit $\bar{p} = (p_s^S, \dots, p_0^S, p_0^C, \dots, p_t^C)$ un chemin dans l'espace des matériaux, on appelle $\bar{p}^S = (p_s^S, \dots, p_1^S)$ le sous-chemin vers la source, il connecte p_0^S à une source de lumière, et $\bar{p}^C = (p_1^C, \dots, p_t^C)$ le sous-chemin vers la caméra, il connecte p_0^C à la caméra.

Nous allons réécrire l'intégrale de bordure de manière à, pour un point de discontinuité entre p_0^S et p_0^C , prendre en compte le sous-chemin de source et le sous-chemin de caméra :

$$\int_{\partial\Omega} \hat{f}^S \hat{f}^B \hat{f}^C \quad (32)$$

avec :

$$\hat{f}^B := \Delta G(x_0^S \leftrightarrow x_0^C) V_{\Delta\mathcal{B}} \quad (33)$$

$$\hat{f}^S := \hat{f}_s(p_1^S \rightarrow p_0^S \rightarrow p_0^C) \prod_{n=1}^s \hat{f}_s(p_{n+1}^S \rightarrow p_n^S \rightarrow p_{n-1}^S) G(x_{n-1}^S \leftrightarrow x_n^S) \quad (34)$$

$$\hat{f}^C := \hat{f}_s(p_0^S \rightarrow p_0^C \rightarrow p_1^C) \prod_{n=1}^s \hat{f}_s(p_{n-1}^C \rightarrow p_n^C \rightarrow p_{n+1}^C) G(x_{n-1}^C \leftrightarrow x_n^C) \quad (35)$$

Nous pouvons réécrire cela sous la forme :

$$\int_{\mathcal{B}} \int_{\Delta\mathcal{B}} \left[\int_{\hat{\Omega}} \hat{f}^S d\mu(\bar{p}^S) \right] \hat{f}^B \left[\int_{\hat{\Omega}} \hat{f}^C d\mu(\bar{p}^C) \right] d\ell(p_0^C) dA(p_0^S) \quad (36)$$

Pour pouvoir échantillonner un point sur la bordure, nous allons effectuer un changement de variable pour passer de p_0^S et p_0^C à x^B le point de discontinuité, et $\omega^B := x_0^S \rightarrow x_0^C$:

$$\iiint \left[\int_{\hat{\Omega}} \hat{f}^S d\mu(\bar{p}^S) \right] \hat{f}^B J^B(x^B, \omega^B) \left[\int_{\hat{\Omega}} \hat{f}^C d\mu(\bar{p}^C) \right] d\omega^B dx^B \quad (37)$$

où

$$J^B(x^B, \omega^B) = \left| \frac{dA(x_0^S) d\ell(x_0^C)}{dx^B d\omega^B} \right| \left\| \frac{dA(p_0^S) d\ell(p_0^C)}{dA(x_0^S) d\ell(x_0^C)} \right\|$$

Grâce à cela nous pouvons créer l'estimateur de Monte Carlo suivant :

Algorithm 4 . – Differentiation du Path Tracer : bordure

```

1: procedure PT_diff_intérieur( $\mathbb{P}$ , Bool direct)
2:   ▷ Échantillonnage d'un segment de bordure
3:    $(x^B, \omega^B) \leftarrow (x, \omega) \sim \mathbb{P}(x, \omega)$ 
4:    $\text{intersec}_{\text{src}} \leftarrow \text{Intersection}((x^B, -\omega^B))$ 
5:    $\text{intersec}_{\text{cam}} \leftarrow \text{Intersection}((x^B, \omega^B))$ 
6:   ▷ Si les deux rayons tapent
7:   if  $\text{intersec}_{\text{src}} \wedge \text{intersec}_{\text{cam}}$  then
8:      $x_0^S \leftarrow \text{intersec}_{\text{src}}.\text{point}$ 
9:      $x_0^C \leftarrow \text{intersec}_{\text{cam}}.\text{point}$ 
10:
11:      $\beta^B \leftarrow \hat{f}^B J^B(x^B, \omega^B) / \mathbb{P}(x^B, \omega^B)$ 
12:      $p_0^S \leftarrow P(x_0^S, \pi)$ 
13:      $p_0^C \leftarrow P(x_0^C, \pi)$ 
14:     ▷ Échantillonnage d'un sous chemin
15:     if direct then
16:        $\beta^S \leftarrow \text{intersec}_{\text{src}}.L_e(x_0^S \rightarrow x_0^C)$ 
17:     else
18:        $\beta^S \leftarrow \text{Estimation du chemin source}(p_0^C, p_0^S)$ 
19:     end
20:      $\beta^S \leftarrow \text{Estimation du chemin de la camera}(p_0^C, p_0^S)$ 
21:     return  $\beta^S \beta^B \beta^S$ 
22:   else
23:     return 0
24:   end
25: end

```

Pour l'échantillonnage du point de discontinuité et de la direction, nous pourrions utiliser une distribution uniforme, mais cela pourrait donner une convergence lente. C'est pour cela que nous pouvons utiliser une carte de photons (issue du photon mapping) et une carte d'importance (issue d'un photon mapping partant de la caméra). Pour voir plus en détail comment nous pouvons mettre en place cet échantillonnage, veuillez consulter le travail de *Shuang Zhao* et son équipe (**Path-Space Differentiable Rendering** [1]).

3.3 | Différentiation des estimateurs

Contrairement à *Shuang Zhao* et son équipe, *Tizian Zeltner*, *Sébastien Speierer*, *Iliyan Georgiev* et *Wenzel Jakob* ont proposé une méthode pour calculer la différentielle sans utiliser le calcul du rendu en lui-même. Pour cela ils ont créé plusieurs estimateurs en appliquant différentes méthodes dans différents ordres. Cela leur a permis d'obtenir des estimateurs « détachés » et « attachés » au paramètre π de la scène. Ainsi, en combinant ces méthodes à l'aide du multi-importance sampling, qui consiste à effectuer plusieurs méthodes d'échantillonnage et à les additionner en leur attribuant un certain poids en fonction de leur efficacité, ils parviennent

à calculer la dérivée de la scène. Pour plus d'informations, veuillez consulter **Monte Carlo Estimators for Differential Light Transport** [5].

4 | Implantation des algorithmes

Afin de pouvoir comprendre les algorithmes étudiés, nous avons implanté un ray tracer et un path tracer.

4.1 | Premier ray tracer

Nous avons donc commencé par suivre une série de tutoriels : **Ray Tracing in One Weekend — The Book Series** [2], [3], [4], écrit principalement par *Peter Shirley*.

Le but de ces tutoriels était d'implanter, en C++, un premier ray tracer.

Cela nous a permis de comprendre le ray tracing de façon concrète, nous avons écrit plusieurs abstractions (formes, matériaux, textures, caméra, *ect.*), qui nous ont ensuite permis d'appliquer les mathématiques de l'algorithme.

Cette première implantation souffrait de la faible parallélisation du CPU. En effet, les temps de rendu pouvaient être très longs.

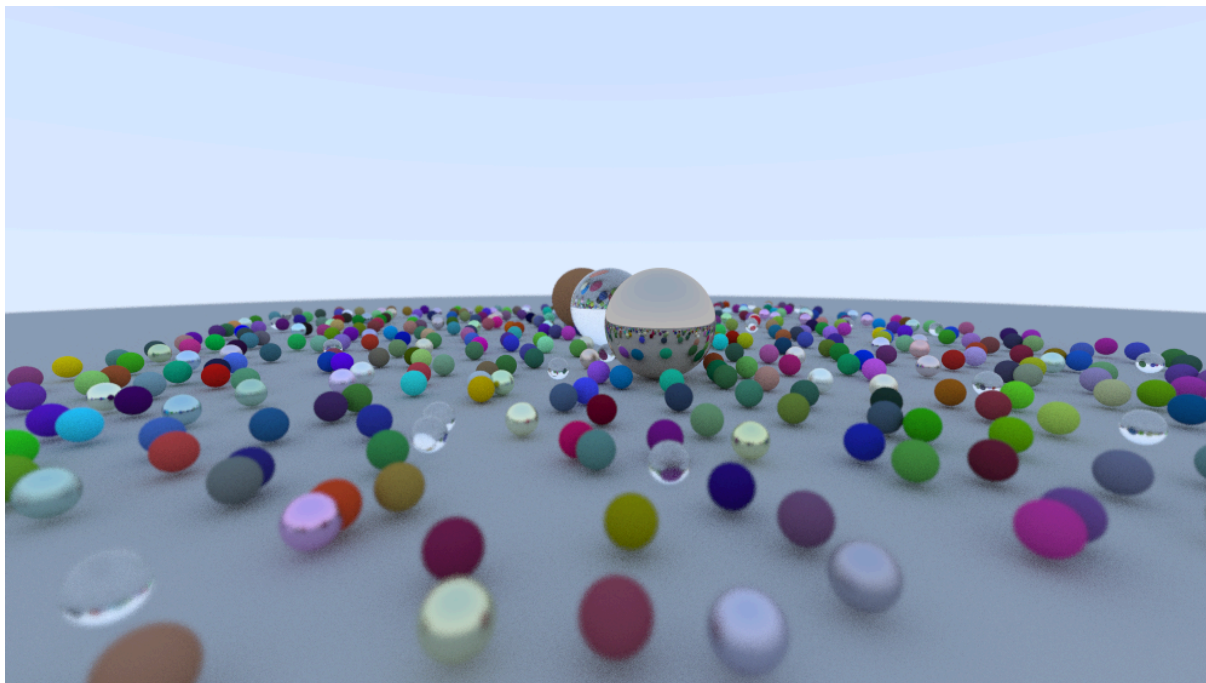


Fig. 3 . – Scène rendue à partir du ray tracer implanté, le temps de rendu est d'environ 3 minutes (CPU \$3.5 GHz\$).

4.2 | Autres implantation CPU

La suite des tutoriels de *Peter Shirley* nous a amenés à construire des rendus plus élaborés. Nous avons donc ajouté la possibilité d'avoir des matériaux volumétriques et des textures.

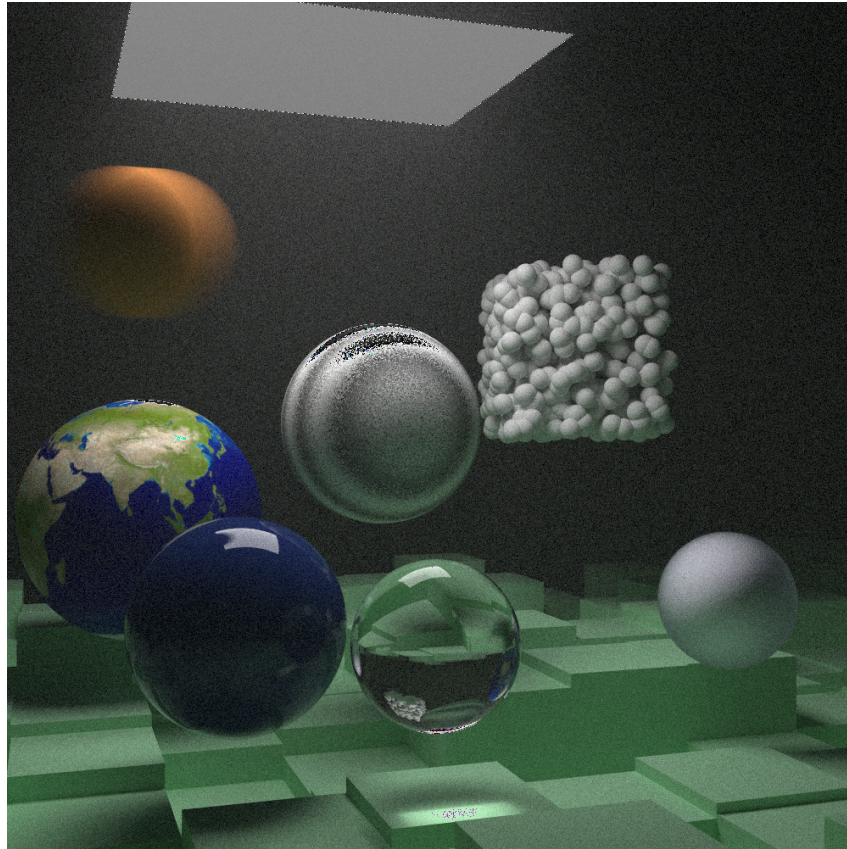


Fig. 4 . — Scène rendue à partir du ray tracer CPU implanté.

Afin d'accélérer le processus, nous avons implanté divers moyens d'accélérer le processus, comme des **BVH** (Bounding Volume Hierarchy, comprendre hiérarchie des volumes englobants), et des méthodes de Monte Carlo plus efficaces, par exemple, au lieu d'opérer la quadrature sur une marche aléatoire à direction uniforme, nous tirons une direction en fonction de la distribution de la **BRDF**.

Au fur et à mesure, le tutoriel nous amène à construire un path tracer, les nouvelles directions sont donc tirées en fonction du placement des lumières, et l'on passe d'un programme récursif à terminal-récursif.

De plus, nous avons ajouté quelques fonctionnalités supplémentaires, telles que le chargement de maillage et le multi-threading.

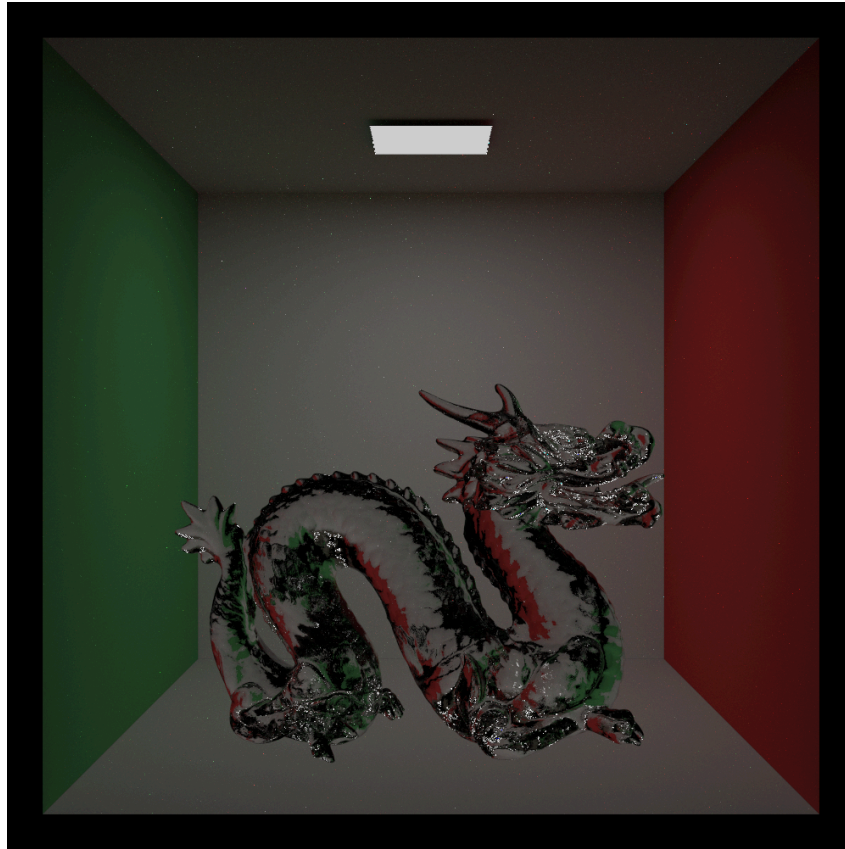


Fig. 5 . – Scène rendue à partir du path tracer CPU implanté.

4.3 | Passage sur GPU

Pour la suite, nous avons décidé d'utiliser Vulkan pour une implantation sur GPU. Vulkan est une spécification proposée par *Khronos Group* (qui propose aussi OpenGL), qui a vocation à remplacer OpenGL. Vulkan nous permet d'avoir un grand contrôle sur la carte graphique, mais comme le dit l'adage : « Un grand pouvoir implique de grandes responsabilités », la spécification est donc dure à prendre en main, avec un débogage compliqué, et une quantité de code à écrire conséquente.

Afin de nous aider, nous avons encore une fois suivi ce tutoriel : **VulkanGuide** [14], qui nous a permis d'implanter un simple moteur graphique de rasterization.

Beaucoup d'abstractions utilisées dans le tutoriel ont été réutilisées par la suite.

Une fois Vulkan appréhendé, nous avons fait notre premier ray tracer sur GPU, bien que rudimentaire, ce dernier était beaucoup plus rapide que les implantations sur CPU.

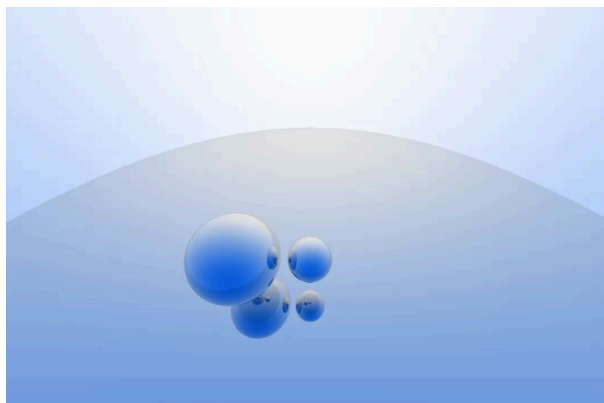


Fig. 6 . – Scène rendue à partir du ray tracer fait sur `Vulkan`, rendu en temps réel (carte graphique intégrée sur un CPU).

Afin de structurer le code, nous avons lu (en partie) le livre **Physically Based Rendering: From Theory To Implementation** [15], qui aborde l'implantation complète d'un moteur de rendu efficace, avec une approche moderne (en C++).

De plus, nous avons utilisé le langage de shader **Slang** [16], un langage de shader qui nous permet d'exécuter du code directement sur la carte graphique. Nous avons choisi ce langage car l'auto-différentiation y est implanté.

Malheureusement, le temps nous a manqué, l'implantation du ray tracer sur Vulkan a pris plus de temps que prévu, et la complexité des mathématiques utilisées dans l'algorithme du path tracer nous a ralenti. Le path tracer que nous voulions implanter n'a donc pas pu être fini à temps.

Nous avons donc décidé qu'après le rendu de ce rapport, nous finaliserons l'implantation, pour à terme, y inclure les travaux de *Shuang Zhao*.

Le code pourra être trouvé sur le repo Github **Vulkan Path Tracer**.

5 | Conclusion

Au terme de ce TER nous avons donc vu la théorie derrière le rendu physiquement réaliste, notamment avec l'équation du rendu [6], et les abstractions mathématiques qui l'accompagnent. Nous avons ainsi pu mettre en œuvre cette théorie, grâce aux algorithmes du ray tracing [17] et du path tracing [7], en l'appliquant sur CPU et partiellement sur GPU, afin d'accélérer les temps de calcul. De plus, nous avons étudié différentes méthodes de différentiation du rendu, celle de *Zhao*, qui sépare le résultat en deux problèmes différents, avec pour le premier une simple différentiation des calculs du path tracer, et pour le second une méthode à part pour les segments de discontinuité de l'image.

Ensuite nous avons vu une méthode alternative, proposée par *Wenzel*, où la méthode consiste à différencier les estimateurs.

Notre formation (MIDL à l'Université de Toulouse) nous a permis d'acquérir une solide base de connaissances permettant de faire face aux problèmes et aux solutions auxquels nous avons été confrontés durant ce stage, qui s'avérait être un excellent pont entre la théorie mathématique et ses implications en informatique. Plusieurs domaines étudiés nous ont été particulièrement utiles, comme les probabilités, l'analyse, l'algorithmique et la programmation étudiées lors de

notre parcours, même si une base en théorie de la mesure et en programmation sur GPU aurait pu nous permettre une compréhension plus rapide de certains concepts.

Par la suite, nous aimerions terminer notre implantation sur GPU, notamment celle du path tracer et la méthode développée par *Zhao*. Et de plus continuer l'étude du rendu différentiel. En effet, l'équipe de *Shuang Zhao* a peaufiné sa méthode, pour l'étendre aux milieux semi-transparents (gaz/liquides), et a amélioré l'algorithme. Il faut aussi noter que d'autres méthodes sont en cours de développement, par exemple des estimateurs de fonctionnelles développés à l'IRIT.

6 | Annexe

Le code source de nos travaux est trouvable sur trois dépôts GitHub différents :

- Le code sources des tutoriels, et des implantations CPU sur https://github.com/CorentinVaillant/travaux_TER.git
 - L'implantation du path tracer sur GPU est disponible sur <https://github.com/CorentinVaillant/Vk-Path-Tracer>
- Et le rapport se trouve sur https://github.com/JulienLEFEBV/Rapport_TER

Bibliographie

- [1] C. Zhang, B. Miller, K. Yan, I. Gkioulekas, et S. Zhao, « Path-Space Differentiable Rendering », *ACM Trans. Graph.*, vol. 39, n° 4, p. 143:1-143:19, 2020.
- [2] P. Shirley, T. D. Black, et S. Hollasch, *Ray Tracing in One Weekend*, 4^e éd. 2025. [En ligne]. Disponible sur: <https://raytracing.github.io/books/RayTracingInOneWeekend.html>
- [3] P. Shirley, T. D. Black, et Steve Hollasch, *Ray Tracing: The Next Week*, 4^e éd. 2025. [En ligne]. Disponible sur: <https://raytracing.github.io/books/RayTracingTheNextWeek.html>
- [4] P. Shirley, T. D. Black, et Steve Hollasch, *Ray Tracing: The Rest of Your Life*, 4^e éd. 2025. [En ligne]. Disponible sur: <https://raytracing.github.io/books/RayTracingTheRestOfYourLife.html>
- [5] T. Zeltner, S. Speierer, I. Georgiev, et W. Jakob, « Monte Carlo Estimators for Differential Light Transport », *Transactions on Graphics (Proceedings of SIGGRAPH)*, vol. 40, n° 4, août 2021, doi: 10.1145/3450626.3459807.
- [6] J. T. Kajiya, « The rendering equation », *SIGGRAPH Comput. Graph.*, vol. 20, n° 4, p. 143-150, août 1986, doi: 10.1145/15886.15902.
- [7] E. Veach, « Robust monte carlo methods for light transport simulation », Doctoral dissertation, Stanford University, Stanford, CA, USA, 1998.
- [8] Wikipedia, « Metropolis light transport — Wikipedia, The Free Encyclopedia ». 2026.
- [9] I. Georgiev, « Implementing Vertex Connection and Merging », technical report, 2012.
- [10] Wikipedia, « Cone tracing — Wikipedia, The Free Encyclopedia ». 2026.
- [11] Wikipedia, « Gaussian splatting — Wikipedia, The Free Encyclopedia ». 2026.
- [12] P. CERMELLI, E. FRIED, et M. E. GURTIN, « Transport relations for surface integrals arising in the formulation of balance laws for evolving fluid interfaces », *Journal of Fluid Mechanics*, vol. 544, p. 339-351, 2005, doi: 10.1017/S0022112005006695.
- [13] O. Reynolds, M.A, et F.R.S, « The Sub-Mechanics Of The Universe », *University Press, Cambridge*, vol. 3, p. 17-254, 1903, [En ligne]. Disponible sur: <https://academicweb.nd.edu/~powers/ame.20231/reynolds1903.pdf>
- [14] P. Marsceill, « Vulkan Guide », 2025, [En ligne]. Disponible sur: <https://vkguide.dev/>
- [15] Matt Pharr Wenzel Jakob et G. Humphreys, *Physically Based Rendering: From Theory To Implementation*, 4^e éd. 2023. [En ligne]. Disponible sur: <https://pbr-book.org/4ed/contents>
- [16] « The Slang Shading Language and Compiler ». 9450 SW Gemini Drive #45043 Beaverton, OR 97008 United States, 2026.
- [17] T. Whitted, « An improved illumination model for shaded display », *Commun. ACM*, vol. 23, n° 6, p. 343-349, juin 1980, doi: 10.1145/358876.358882.